

fleetsim: a validated discrete-event simulator for machine-learning accelerator fleets

Felipe Valencia

<https://github.com/felivalencia3/mini-compute-simulator>

July 30, 2026

Abstract

fleetsim is an open-source discrete-event simulator for fleets of machine-learning accelerators. It models a hierarchy of clusters, pods, racks and nodes; gang scheduling, where a job needs all its chips at once or none; preemption priced in the training work it destroys; node failures, repair and maintenance drains; and a traffic generator whose parameters come from published cluster characterizations. Existing open tools sit at other *altitudes* — altitude meaning what one simulated run represents: a single cluster with a flat GPU count, or one job’s network traffic with no queue in front of it — and the production fleet managers at this altitude ship nothing anyone outside can run.

The central claim is validation against published measurements of real clusters. On the four Helios GPU clusters, **fleetsim** reproduces the published ratio between first-in-first-out (FIFO) and shortest-job-first mean completion times inside the stated tolerance bands on all four, at a four-cluster mean absolute error of 3.4 %. The absolute mean FIFO completion time on Saturn, the busiest of the four, lands within 0.01 % of published — a coincidence, not a precision claim, because reordering jobs inside the trace’s tied submit seconds moves that same quantity by 17 %. What the absolute comparison supports is agreement within ± 14 % across all four clusters, never a single cell. Two further qualifiers travel with the result: the bands are wide for stated reasons — a deliberately bad placement policy passes them — and every number holds only under three named modelling choices. We also list the published quantities **fleetsim** deliberately does *not* reproduce.

Availability. **fleetsim** is Apache-2.0 licensed at

<https://github.com/felivalencia3/mini-compute-simulator>

The offline checks are one command, `pytest tests validation -q`. The Helios replay is environment-gated: it downloads the 36 MB public trace and replays four clusters under two schedulers in roughly four minutes (250–266 s across the repository’s own timings, on a laptop).

1 Introduction

A fleet of machine-learning accelerators is a large capital asset that behaves like a queueing system. The chips are bought in bulk, wired into a fixed hierarchy, and handed out — minute by minute, for months — by one program: the scheduler, which decides which queued job runs now and which waits.

That decision has more leverage than it looks. On a 2,048-chip fleet — the shipped `examples/01_minimal` scenario, not one of the case studies in §6 — replacing strict first-in-first-out (FIFO) queueing with a preempting tiered-priority policy cuts the mean number of waiting evaluation jobs from about 11 to about 1.3, and their 99th-percentile queue wait from about 5 h to

about 31 min. Same hardware, same arrivals, same seed; a different ordering rule. (Those are the figures the example’s own README records, to the precision it records them.)

You cannot run that experiment on the production fleet: an A/B test costs weeks of accelerator time, and traces record what one scheduler did, never what another would have done.

fleetsim is an open-source discrete-event simulator that answers such questions at *fleet altitude*. Altitude here means what one simulated run represents. One **fleetsim** run is weeks of a whole fleet: thousands of jobs arriving, queueing, running, being preempted and occasionally dying with their nodes. A network simulator’s run, by contrast, is one already-placed job executing a single training step. Given a fleet in YAML, arriving jobs and a failure model, **fleetsim** reports the occupancy, queue waits, preemption rates and goodput a policy delivers — four quantities §2.3 and §2.4 define, and whose differences turn out to matter more than any one of them alone. It models no packets, collectives or kernels — per-job speed is one analytical multiplier — because its questions concern thousands of jobs contending for one machine, not one job’s step time.

We claim three contributions, in ascending order of importance.

1. **A fleet-altitude model** (§3): one capacity tree from metro down to node, spanning NVIDIA-style clusters and TPU-style pods, with gang scheduling, locality constraints, preemption cost, quota, failures and drains in one engine; the simulators we surveyed each model a subset (§8).
2. **A generative traffic model** (§4): arrivals, job sizes and durations from published cluster characterizations rather than a single Poisson assumption, each parameter recorded with its source — including those that are order-of-magnitude priors rather than fitted constants.
3. **A validation suite** (§5) that replays a real production trace — the four Helios clusters [5] — and reproduces the published FIFO-versus-shortest-job-first completion-time ratios to a mean absolute error of 3.4%, under three stated modelling choices we restate wherever the result appears. It adds closed-form queueing checks, a list of published numbers **fleetsim** deliberately does *not* reproduce, and one mechanism we had misdiagnosed (§5.3).

§2 primes readers new to this infrastructure; §3–4 give the model and traffic generator, §5 the validation, §6–9 the case studies, implementation, related work and limitations.

2 Background: what a fleet is, and why scheduling it is hard

2.1 The physical machine

The unit of capacity is a *chip*: one GPU or one TPU. Eight typically share a server chassis and a fast internal switch — a *node*. Tens of nodes make a rack or scalable unit, and tens of those a *pod*: 4,096 GPUs in the H100 configurations we model, and 4,096 chips in a TPU v4 pod [8]. Pods join into *clusters*, then datacenters and *metros*.

Every level is a bandwidth cliff, which is why locality matters. Llama 3’s cluster was eight pods of 3,072 GPUs at 1:7 oversubscription, and the 405B run took 16,384 of those GPUs [12]. The 1:7 figure means seven times less bandwidth between pods than within one, and a job straddling such a boundary runs slower for its whole life.

2.2 Gang scheduling and fragmentation

A distributed training job is not a web service: its chips run one computation in lockstep, synchronizing every step, so it needs *all* of them *at once* for its whole life. Half a training job does not do

half the work; it does none. Allocation is therefore all-or-nothing — *gang scheduling*. The analogy is a restaurant booking for twelve: a table for eight plus a table for four across the street does not seat twelve.

The gang is also the unit of failure: if any node holding part of it dies, the whole job dies. And gangs create *fragmentation*, free capacity that cannot be used: a fleet can have 200 chips free and still not start a 64-chip job, because those chips sit in ones and twos across many nodes. Philly measurements attribute 59–98 % of multi-GPU queueing delay to fragmentation rather than fair-share policy [7]; reproducing that attribution is itself an explicit non-goal (§5.5).

Three kinds of job share the machine, and they are not variations on a theme. A *pretraining* run trains a model from scratch: thousands of chips held in lockstep for weeks. A *finetune* adapts an already-trained model to a narrower task: tens of chips for hours. An *evaluation* scores an existing model on a test set: one or two chips for minutes. All three arrive into the same queue, and they differ by about four orders of magnitude in both size and duration.

That disparity shows up in every trace. Over 82 % of Philly’s jobs use one GPU [7], and 70.8 % of jobs on the busiest Helios cluster we replay are single-GPU. Acme measured evaluations at about 93 % of job *count* and under 1 % of GPU-*time*, with pretraining at 1–3 % of count and 70–94 % of GPU-time [6]. The tiny jobs strand exactly the capacity the enormous ones need.

This is the *mice-and-hogs* shape: a swarm of one- and two-chip jobs (the mice) that dominate the job count, and a handful of thousand-chip jobs (the hogs) that dominate the chip-hours. Almost every metric in this paper has to be reported twice because of it. Count one job as one job and you are describing the mice; weight by chip-hours and you are describing the hogs.

A fleet is also shared. Each *tenant* — a team or project with its own budget — is granted a *quota*, a promised share of the chips the scheduler is expected to honour before it hands anything to anyone else. Quota is a promise about capacity, not a physical partition: the chips themselves stay in one pool.

2.3 Three numbers that are not the same number

One term first, because the third definition below depends on it. A long training job periodically writes its entire state to storage — a *checkpoint* — so that a job killed by a failure or a preemption resumes from the last saved state rather than from the beginning. Saving costs the job real time (about a minute, on *fleetsim*’s default one-hour interval), and everything computed since the last checkpoint is lost when the job dies. The analogy is a document editor that autosaves every hour: a crash costs you at most an hour, and the autosaves themselves cost you a little throughput.

Conflating utilization metrics is the classic error here, so *fleetsim* defines three and always reports all three. *Allocation rate* is allocated chip-time over *total* chip-time, including chips that are broken or drained. *Occupancy* is allocated chip-time over *healthy* chip-time. *Goodput* is the fraction of allocated chip-time that produced surviving work, excluding work lost back to the last checkpoint, restart latency and checkpoint-save overhead.

A worked example. Of 1,000 chips, fifty are down for repair, so 950 are healthy, and all 950 are allocated. Allocation rate is $950/1000 = 0.95$; occupancy is $950/950 = 1.00$, and the gap is exactly the capacity lost to failures — which is why occupancy alone makes a fleet with broken hardware look perfect. Now suppose a third of the allocated chip-time goes on redoing work lost at the last interruption and on restarts: goodput is 0.67, and the share of the machine doing useful work is $0.95 \times 0.67 \approx 0.64$. Fully occupied, a third of its work wasted. Philly clusters were in fact measured at roughly 100 % allocation with about 52 % hardware utilization [7].

2.4 Failures and preemption

At small scale hardware failure is a nuisance; at fleet scale it is a design constraint, because a gang fails when *any* of its nodes fails. A gang of N nodes therefore breaks N times as often as one node does.

Two published anchors bracket the numbers. Meta measured node mean time between failures (MTBF) at about 42 node-days [15]: one node in the fleet breaks about every 42 days. Llama 3’s 405B run, 16,384 GPUs and so about 2,048 nodes, saw roughly one unplanned interruption every 3.1 hours [12]. Scaling that measurement, a gang eight times larger — 16,384 nodes, the size our frontier case study uses — would be interrupted eight times as often, about every 23 minutes.

The two anchors do not agree with each other, and we say so rather than picking the convenient one. Applying the 42-node-day MTBF directly to Llama 3’s 2,048 nodes predicts an interruption every 30 minutes, roughly six times more often than Llama 3 actually reported. The MTBF checks out against Meta’s own smaller figure (42 node-days over the 128 nodes of a 1,024-GPU job gives 7.9 hours, which is what Meta measured) and not against Llama 3’s. `fleetsim` therefore ships 42 node-days as a configurable default, not as a fitted constant, and the 23-minute figure quoted later comes from scaling Llama 3’s measurement, not from the MTBF.

Preemption — stopping a running job to give its chips to a more important one — costs the same way: a preempted training job does not pause, it loses everything since its last checkpoint and pays about 15 minutes of restart overhead [15]. Under a one-hour checkpoint interval that is up to an hour of work per chip in the gang. This is how aggressive preemption raises occupancy while destroying goodput.

3 What fleetsim models

3.1 One tree for two ecosystems

Capacity is a tree of *domains* from metro down to node, all one `Domain` class. Leaves are nodes carrying a chip count, a chip type, a health state (`HEALTHY`, `DRAINING`, `FAILED`, `MAINTENANCE`) and a failure-rate multiplier. Interior domains carry chip counters maintained incrementally, making “how much free capacity is under this domain” an $O(1)$ query.

One tree covers both ecosystems because *level names are configuration, not code*: a cluster declares `[cluster, pod, su, node]` for an H100 SuperPOD or `[cluster, pod, cube, host]` for a TPU v5p pod. The scheduler sees domains, capacities and constraints, never vendors.

3.2 The allocation rule, and why it is the hardest call

Jobs are allocated chip-granularly *within* a single node and in whole-node multiples *above* one node: a 2-chip evaluation may share a node, a 64-chip job asks for eight entire nodes. The reason is workload shape — `fleetsim`’s shipped class mix puts about 90 % of arrivals in 1–8-chip evaluations, following the Acme characterization [6], which measured the corresponding share at about 93 %. Were the node atomic, a 1-chip evaluation would strand seven GPUs and corrupt both occupancy and the mice-and-hogs story; rounding those jobs up would falsify the workload.

The rule has one load-bearing consequence, the mechanism behind our main validation finding: *a leaf with any owner at all is invisible to every whole-node request*. A node with one chip in use has seven free chips no multi-node gang can claim, so free capacity accumulates as 1–7-chip remainders. That whole-node rule is an approximation we measured: in the Helios September window the trace’s own node count satisfies `node_num = ⌈gpu_num/8⌉` for 99.66 % of multi-GPU jobs, and all 225 exceptions used *more* nodes than the ceiling, never fewer — the half the model depends on.

An allocation is atomic, immutable once granted, and the unit of failure. A gang may name a level it must fit *within*, and may be *segmented* into whole-node blocks that each sit inside one domain at that level.

3.3 The engine

Time is an `int64` count of microseconds, which makes tie-breaking exact. The engine is event-driven rather than tick-driven because state changes are sparse and unevenly spaced: a fleet can go minutes without one, then process hundreds of preemptions at a single instant.

Events sit in a heap keyed by `(time, type, seq)`. The event type is not decoration — its integer value *is* the same-timestamp ordering rank, arranged so every repair, completion, failure and preemption at time t is processed before the scheduler wakes at t , so it always sees settled state. Scheduler wakes are then *coalesced*: events set a dirty flag and ensure at most one pending wake at the next round boundary, so the scheduler runs once per round (60s by default) rather than once per event.

3.4 Schedulers and placement

Ordering and placement are separate axes [4]: a scheduler sees an immutable view of the cluster and returns `Place` and `Preempt` intents, which the engine validates and applies [3].

Four schedulers ship. `fifo` serves the queue in arrival order, with strict head-of-line blocking by default: the head of the queue holds up everything behind it until it can run. `tiered_priority` implements Borg-style priority bands [20, 19] with requeue preemption. `sjf` runs the shortest job first. `easy_backfill` adds *backfilling* [11]: letting a small job at the back of the queue jump ahead into a hole the head job cannot use, on the condition that doing so provably does not delay the head. That condition is computed from each job’s declared runtime estimate, and users routinely over-estimate, so a backfill scheduler must behave sanely when the estimate is wrong. Plugins register through a Python entry point.

Orthogonally, four placement policies choose *which* fitting nodes to take. `first_fit` (the default) takes the first nodes that fit, scanning by ascending node identifier. `best_fit` takes the nodes with the least free space that still fits, so partly-used nodes fill up before empty ones are opened — *tightest-fit* packing, a term used throughout the rest of the paper. `consolidate` does the same at the sub-node level and additionally minimises how many parent domains a multi-node gang touches; on a single-level fleet it is bit-identical to `best_fit`. `spread` takes the emptiest nodes (worst fit) — a deliberate anti-policy, our control arm.

3.5 Failures, repair and maintenance

Failures are sampled by *aggregate* rate — one exponential draw at the fleet-wide rate, then a victim node — not by per-node timers, so a job’s mean time to interruption scales inversely with its size for free.

Defaults are sourced, not invented: node MTBF 42 node-days, automatic repair uniform on 60–180 minutes, 10% of failures needing manual repair over 1–3 days, one maintenance drain per node-month, a one-hour drain grace window, and a failure-cause mix of ~60% GPU/HBM, ~10% network and ~13% software, with the remaining 17% a residual “other” bucket `fleetsim` adds so the mix sums to one. The MTBF comes from Meta’s measurements [15], the three approximate cause shares from Llama 3 [12], and the drain rate from Borg [20]. The repair distributions are the weakest-sourced of the set: the repository attributes them to AIReSim [1], a reliability simulator it names but does not pin to a citable artefact, so a reader cannot check them the way the others can.

Table 1: Metric definitions. “Allocated” means chips held by a running job; “healthy” excludes nodes that are failed, draining or under maintenance. Integrals are exact integer chip-microseconds internally, reported as chip-seconds.

Metric	Definition	What it tells you
Allocation rate	$\int \text{allocated } dt / \int \text{total } dt$	Share of the purchased fleet handed out
Occupancy	$\int \text{allocated } dt / \int \text{healthy } dt$	Occupancy minus allocation rate is capacity lost to failures and drains
Goodput	$\sum \text{productive chip-s} / \int \text{allocated } dt$	Excludes work lost since the last checkpoint, restart latency and checkpoint-save overhead
ETTR (per job)	Effective training time ratio: productive seconds / wall seconds holding an allocation	Goodput seen from one job’s side; counts restart overhead and preemption grace in the denominator
Fragmentation index	$1 - \text{largest placeable} / \text{total free at a level}$	0 when all free capacity is usable by one job
Stranded whole nodes	healthy leaves with $0 < \text{free} < \text{chips}$	Free capacity no whole-node gang can claim

All are configuration values, none universal. Maintenance is a separate stream: a **DRAINING** node takes no new placements, and its residents are requeued when the grace window expires.

3.6 Metrics

Table 1 states the definitions. State metrics are time-weighted integrals updated on every state change — exact, not sampled — and accumulated as integer chip-microseconds; discrete outcomes are event-sourced.

Two conventions matter. Metrics are given over the full run and over a steady-state window (the middle 80 % of the horizon by default), and every distributional metric is reported job-weighted *and* chip-hour-weighted — a job-weighted mean over a mice-dominated workload describes the mice. Best-effort jobs are excluded from wait distributions. *Best-effort* is the lowest, always-preemptible priority band, and **fleetsim** submits it from a *closed loop*: a source that keeps a fixed number of jobs in flight and launches a replacement the instant one finishes. A closed loop by construction always has work queued, so its queue length says nothing about the fleet and its mean waiting time is undefined. Best-effort work is therefore reported by its goodput instead.

4 Modelling the traffic

Much of what a fleet simulator tells you depends on what arrives at it. The textbook default is Poisson arrivals: jobs appearing independently of one another at a constant average rate, with exponentially distributed gaps. Poisson traffic is smooth in a specific sense — average it over a longer window and its relative variability shrinks predictably. Measured computer traffic does not do that. It is bursty at every timescale you look at, and averaging does not smooth it out [16]. A study built on Poisson arrivals understates queue variance, and understates it most exactly where the answer

Table 2: Arrival process per job class, with shipped defaults. Pretraining is the one surviving Poisson case: rare planned launches really are independent, human-initiated events.

Class	Process	Defaults
Pretrain	homogeneous Poisson	<code>rate_per_week: 2</code>
Finetune	MMPP-2 $\times$ diurnal envelope	<code>rate_ratio: 4, burst_frac: 0.25, switch_tau: 2d</code>
Eval	Hawkes on an NHPP baseline	<code>branching: 0.4, kernel_tau: 15m</code>
Inference	service replica curve	—
Best-effort	closed-loop backlog	<code>concurrency: 4, think_time: fixed[0s]</code>
Retry storm	near-critical Hawkes (preset)	<code>branching: 0.9, kernel_tau: 2m</code>

matters, near saturation. `fleetsim` therefore runs one arrival process per class and superposes them (Table 2).

The shared day-and-week machinery is a non-homogeneous Poisson process whose log rate is a sum of harmonics at 24h and 168h. The exponential keeps the rate positive, and the constant term is derived rather than configured, so changing the shape of a day does not change the amount of work. The shipped daily preset is a two-harmonic least-squares fit to `fleetsim`’s own v0.1 step curve, which was itself built from the Helios description of a night trough with dips at 12:00 and 18:00 [5]. The provenance matters because the fit loses something: two harmonics smooth those two dips away, leaving a trough near $0.59\times$ at 03:00 and a single peak near $1.27\times$ at 10:15. The shipped preset is therefore continuous with the previous release’s behaviour rather than a fit to a published curve, and it no longer reproduces the midday and evening dips Helios reports. It is sampled by Lewis-Shedler thinning — propose arrivals at a constant upper-bound rate, then keep each with probability equal to the true rate over that bound.

Finetuning sits on a two-state Markov-modulated Poisson process: a hidden switch between a quiet and a busy regime, reproducing crunch weeks that a single rate curve cannot. It is configured in observable terms — mean rate, burst fraction, switch correlation time, rate ratio — and solved to state rates in closed form. Those rates must be scaled to the *peak* of the diurnal envelope before thinning; left unscaled they silently generate about 60 % of the configured load.

Evaluation traffic uses a Hawkes process. Self-exciting means one submission makes further submissions more likely for a while afterwards: a failed eval is re-run, a user iterates. Configuration exposes the branching ratio — mean children per arrival, stationary below 1 — and the kernel decay time; sampling uses Ogata thinning, sequential but $O(1)$ per event because the excitation term decays exponentially.

Sizes are per-class distributions over powers of two. Evals are 55 % single-chip: inside the 50–90 % single-GPU range Helios reports, and deliberately below Philly’s finding that over 82 % of jobs are single-GPU [7]. TPU clusters use the published histogram of *slice* sizes [8] — a TPU slice being a contiguous geometric block of chips carved out of a pod, a different thing from the quota slices of §5.2. Uniform-over-exponents sizing remains available but is not the default, since it overweights giants by 5–10 $\times$ and fabricates queueing.

Job durations are drawn from a lognormal shape, configured by naming its quantiles (median and 90th percentile) rather than its abstract parameters. Pretraining additionally gets a *heavy tail*: a small probability of a run lasting far longer than the median, decaying as a power law rather than dying off exponentially. How heavy a tail is, is summarised by its *tail index*; a smaller index means a heavier tail. Pretraining gets the only heavy tail because resource-hours are size $\times$ duration, and a product of two quantities is as heavy-tailed as the heavier of them — it inherits the *smaller* of the two indices. Attaching the tail to the one class with genuinely open-ended runs is therefore

intended to suffice for the Borg-like resource-hour tail [19], without fabricating implausibly long evaluation jobs. Intended, not confirmed: the distributional self-check that would measure the generated chip-hour tail is a documented target in the traffic specification, not a shipped test.

Tenants come from a *finite* Zipf over ranks. An exponent of 1.19 reproduces Alibaba PAI’s top 5 % of users submitting about 77 % of instances [22], at a population of 1,300 tenants; the shipped default is 1.2. The class mix follows Acme [6].

These parameters are sourced but not all fitted, and the repository says so: the Markov-modulated and Hawkes defaults are order-of-magnitude priors borrowed from grid and LLM-serving analogies, not constants fitted to an ML fleet. There is no explicit long-range-dependence generator, tenants are marks on jobs rather than agents, and within a class size and duration are drawn independently.

5 Validation

5.1 What “validated” means here, and the two kinds of check

A simulator can be internally consistent and still be wrong about the world. Validation here means comparing fleetsim’s output against numbers a published study measured on a real cluster, and two kinds of comparison are possible. A **policy-effect check** runs two scheduling policies over the same trace and compares the *ratio* between them against the ratio the paper reported. A **distribution-match check** asks whether a converted trace reproduces a published shape, such as the fraction of jobs that ended successfully; that mostly tests the converter rather than the scheduler, and in the Philly case involves no simulation at all.

The ratio is the more robust test, and the reason deserves two extra sentences. Suppose the fleet we reconstructed is 10 % smaller than the real one. Both policies then run on a fleet 10 % too small, both produce waits that are too long, and much of that shared error divides out when one is taken over the other. An absolute number has nothing to divide it out; it carries every reconstruction error straight through.

Replays never match exactly, for structural reasons. Take our own replay first. Helios publishes its analysis window — 2020-09-01 to 2020-09-26 — and we replay exactly that, so the window is not a source of error here. Two others remain. Timestamps carry no timezone, so a daily cycle cannot be aligned with confidence. And the original scheduler made placement decisions nobody recorded, which is the residual §5.3 spends most of its length on.

Where a released trace and its paper’s analysis window differ, the problem is worse. Philly released 117,325 jobs over 137 days while the paper analysed 96,260 over roughly 75, and that window is not precisely published, so no reconstruction of it can be checked. The tolerance bands below are sized to absorb all of those residuals; they are not calibrated uncertainty intervals.

5.2 Replaying a real trace: the Helios experiment

The flagship replay targets Hu et al.’s Helios study [5], which characterizes four production GPU clusters — Venus, Earth, Saturn and Uranus — over 2020-09-01 to 2020-09-26, GPU jobs only. The trace is a 36,437,672-byte archive (343,069,364 bytes uncompressed) under CC-BY-4.0; fleetsim fetches it, verifies it by size, and replays all four clusters in about four minutes.

Each cluster is partitioned into *virtual clusters*: quota slices, each a group’s private share of the machines, scheduled independently. The reference simulator runs one scheduler per slice, whereas fleetsim schedules a single global pool, so the replay is a harness rather than an engine change — one

Table 3: Two modelling assumptions the validation plan made before the data was touched, and what the real trace showed.

Choice	The plan assumed	What the trace showed
Scan mode	Non-blocking: a job the scheduler cannot place is skipped and later jobs proceed.	The reference blocks. Non-blocking collapses FIFO’s queueing — ratios near $1.4\times$ with the wrong cross-cluster ordering. A strict scan flips the result from non-reproducing to reproducing.
Per-slice capacity	A fixed quota snapshot taken on 1 September.	Quota drifts mid-month: one Uranus slice grows $208 \rightarrow 328$ GPUs, another spins up $0 \rightarrow 416$, and 3,117 jobs sit in slices with no 1 September quota at all, which a snapshot silently drops. It also over-congests Uranus (ratio $3.41\times$), breaking the rank and the queueing-share ordering.

simulation per slice, on a fleet sized to that slice’s node count at 8 GPUs per node, aggregated job-weighted to cluster totals. The converter writes each job’s recorded duration into both its duration and its runtime estimate, making shortest-job-first an *oracle*: a perfect estimate, and so a strict upper bound on what a duration-*predicting* policy could achieve.

Every claim below holds only under three modelling choices, all options a reader could disagree with: a strict blocking scan, September-max per-slice sizing, and **consolidate** placement. Two of them were not chosen but forced — Table 3 records the assumptions the real data disproved. A strict scan means the queue head blocks everyone behind it until it can run, which is what head-of-line blocking means; both policies share the scan mode, so shortest-job-first’s advantage remains purely its ordering. September-max sizing drops zero jobs, and the test asserts that, but it is not unambiguously more faithful and the repository says so: peak quota available from day one over-provisions early in the month and under-counts early-month queueing. The converter’s own default remains the snapshot.

Table 4 gives the result: all eight ratios in band. The quantity being compared is the job completion time, or JCT — the wall-clock time from when a job was submitted to when it finished, so queue wait plus run time. The test also asserts that no job was dropped, that every job reached a terminal state, that Saturn has the highest and Uranus the lowest JCT ratio, and that the queueing share of JCT orders Saturn > Venus > Earth > Uranus in fleetsim (90.1, 79.1, 73.0, 42.2 %) as in the published table (89.7, 81.8, 69.3, 42.5 %). It runs in roughly four minutes on a laptop against the cached trace: the repository’s own timings are 250 s, “about 260 s” and 266 s in three different files, which is the accuracy a wall-clock number of this kind deserves. The bands are wide on purpose, and were deliberately *not* tightened even though every JCT ratio now lands within 5 % of published, for the two reasons in §5.1. Nor is the queueing column an exact reproduction: Earth measures $10.67\times$ against a published $16.4\times$, about 35 % low, and passes only because the band runs to $25\times$. The band does have teeth — SJF replayed against itself gives 1.0 and fails it.

5.3 What the replay found

The explanation we had was impossible. The previous version of the suite had one out-of-band number: Saturn’s JCT ratio at $8.75\times$ against a published $6.59\times$. It also had an explanation — that first-fit placement fragments Saturn’s large multi-node jobs more than the reference implementation’s consolidating placer. That explanation was wrong, and provably rather than merely unsupported. The replay fleet is single-level: every node is a direct child of the cluster root, replay

Table 4: Helios replay, four clusters, real trace, shipped configuration (strict scan, September-max sizing, `consolidate` placement). The `fleetsim` column is the claim. The `first_fit` column is the same replay under the engine’s `default` placer, shown so the size of the placement effect is visible rather than asserted. The *In band* columns judge the `fleetsim` column only. Bands are $[1.3, 8] \times$ on the completion-time ratio and $[3, 25] \times$ on the queuing ratio; † marks the one `first_fit` value that falls outside its band, the deviation §5.3 diagnoses. The mean absolute errors quoted in the text (3.4 % for `fleetsim`, 27.4 % for `first_fit`) are computed on full-precision ratios; the rounded values shown here give 3.2 % and 27.5 %.

Cluster	FIFO/SJF mean job-completion-time ratio				FIFO/SJF mean-queuing ratio			
	Published	<code>fleetsim</code>	In band	<code>first_fit</code>	Published	<code>fleetsim</code>	In band	<code>first_fit</code>
Saturn	6.59	6.87	yes	8.75 [†]	18.5	19.55	yes	22.91
Venus	3.07	3.21	yes	4.21	5.68	7.78	yes	9.97
Earth	2.87	2.95	yes	2.11	16.4	10.67	yes	5.73
Uranus	1.49	1.51	yes	1.69	4.51	5.08	yes	6.10

jobs carry no locality constraint and no segment structure, and no crossing penalty is configured, so *which* free nodes a multi-node job takes cannot change any outcome at all. The sizes agree: Saturn’s largest job is 200 GPUs, and the quota slice carrying 84 % of the FIFO-minus-SJF gap tops out at 56 GPUs, with 0.7 % of its jobs larger than one node. It was never a large-job effect.

The real mechanism: sub-node stranding. One 1-GPU job landing on an empty 8-GPU node is enough. That node now has seven free GPUs and an owner, and the allocator treats any node with an owner as ineligible for a whole-node request — so a 16-GPU gang cannot touch it. Do that on enough nodes and the fleet reports plenty of free chips while no gang can start.

That is what first-fit does here. Saturn is 70.8 % single-GPU jobs, 97.1 % in its dominant quota slice. First-fit scans nodes by ascending identifier with no preference for nodes already partly used, so it opens a fully free node for a 1-GPU job whenever the lower-numbered ones are full, and free capacity accumulates as 1–7-GPU remainders no job of 8 GPUs or more can claim. The whole-node rule itself is an approximation whose error was quantified rather than assumed: Helios’s own node count equals $\lceil \text{gpus} / 8 \rceil$ for 99.66 % of windowed multi-GPU jobs, and every exception spreads a job over *more* nodes than the ceiling, never fewer — “never fewer” being the half the rule depends on.

Instrumented runs confirmed the mechanism: 100 % of measured blocked-idle chip-seconds come from queue heads needing whole nodes, sub-node heads contribute 0.0 %, and 87.7 % of free chips during blocks sit on partly-used nodes. This happens at an *offered load* of 94.4 % — offered load being the arriving work as a fraction of the fleet’s capacity to do it. Near 100 % queues become extremely sensitive: a few percent of capacity made unusable does not lengthen the queue by a few percent, it lengthens it by a multiple [9]. Those instrumentation figures came from throwaway code not in the tree — the diagnostic work that found the mechanism, not numbers the suite regenerates.

What the fix did and did not buy. Switching the sub-node rule to tightest-fit packing — the `consolidate` policy, which fills partly-used nodes before opening empty ones — closes most of the gap. The four-cluster mean absolute error against the published ratios falls from 27.4 % to 3.4 %, and Saturn’s mean FIFO completion time moves from 75,329 s to 55,978 s against a published 55,984 s. That 0.01 % is luck, not precision — the tie-break probe below moves the same quantity by 17 %. What is supported is the band across four clusters in Table 5, never a single cell.

Table 5: Absolute quantities: reported, *not* asserted — continuous integration does not fail if they move. #Queuing counts jobs whose wait exceeds one 60s scheduler round. * marks the two quantities that moved *away* from published when the placement model was corrected.

Cluster	Mean FIFO job completion time (s)			#Queuing jobs		
	Published	fleetsim	Δ	Published	fleetsim	Δ
Saturn	55,984	55,978	−0.01 %	65,991	72,936	+10.5 %
Venus	64,702	55,714	−13.9 %	15,336	13,624	−11.2 %*
Earth	19,754	22,463	+13.7 %	30,030	32,232	+7.3 %
Uranus	19,758	18,492	−6.4 %*	16,917	18,449	+9.1 %

Three qualifications belong with that improvement. One change did not fix everything: Uranus’s absolute FIFO JCT went from +5 % to −6.4 %, overshooting past published rather than approaching it; Venus’s queued-job count drifted from −9 % to −11.2 %; and the published *ranking* of absolute JCTs, which the earlier version happened to reproduce, no longer is — though nobody can meaningfully reproduce it, since published Uranus (19,758s) and Earth (19,754s) differ by 0.02 %, a dead tie. Only the ranking of the *ratios* is asserted. Nor are the eight absolute quantities eight independent confirmations: mean JCT is service plus mean queueing, the ratio is one mean JCT over another, and the queueing share is queueing over JCT, so all are functions of the same per-cluster wait distribution and together are close to one degree of freedom. And the new placer does not dominate the old — of the five Saturn quota slices carrying 97 % of the gap, three get *worse* under it, one by 18.5 %, the cluster-level win being carried by a single slice holding 41 % of Saturn’s jobs. The supported claim is that the policy reproduces the reference placer’s *aggregate* behaviour, never that it is a better scheduler.

The bands do not discriminate between placement policies. *spread*, a deliberately bad control policy that scatters work onto the emptiest nodes, was measured across all four clusters and satisfies every published-facing check above — both bands on all four, both rank invariants, the queueing-share ordering — while being 45 % off Saturn’s absolute FIFO JCT and 99 % off Earth’s. Passing the bands is therefore not by itself evidence that the placement model is right, and “in band” must never be read as “reproduces the paper”. The four-cluster version of that result is a documented measurement; what the suite *asserts* is the Saturn instance of it, so a placement regression fails loudly on one cluster rather than on four. The point of asserting it at all is that the misreading cannot then be made silently.

What selects the placer instead, in descending weight, is: the four-cluster mean absolute ratio error, asserted below 10 % and measuring 3.4 % against 27.4 % for first-fit and 29.5 % for the control, since aggregating four clusters stops any one cluster’s ordering noise from deciding it; a mechanism metric — the time-averaged count of partly-occupied healthy nodes, fleet state rather than a queue statistic — which orders the three policies exactly as their FIFO completion times do; and Saturn’s absolute number, which corroborates but cannot independently discriminate. It cannot because 35.5 % of Saturn’s jobs share an exact submit second, and reversing the tie-break inside those seconds moves Saturn’s FIFO JCT by 17 % under first-fit — enough to invert which placer looks closer to published. That is a sensitivity probe under a deliberately wrong arrival order, not an error bar: ascending identifier is demonstrably the faithful one. Whether the four-cluster discriminator would survive that perturbation is unmeasured. The probe ran on one cluster under one scheduler — Saturn under FIFO, in both tie-break orders — and extending it to all four clusters under both

Table 6: Closed-form and invariant rungs, and what each would catch.

Rung	What it would catch
Erlang-C (M/M/c): mean wait and occupancy within 10 % of analytic	A broken queue discipline, a mis-scaled arrival rate, or a double-counting occupancy integral.
Pollaczek–Khinchine (M/G/1) under lognormal service	Duration-sampler bias the exponential-only rung structurally cannot see, since the formula depends on the second moment of service time (a distribution’s spread, not just its average, drives the wait).
Preemptive-resume priority, two tiers, <i>one server</i> (M/M/1), plus a shielding test under a saturating backlog	Preemption-accounting errors and leakage of low-priority load into high-priority wait. Best-effort mean wait is asserted on nothing — it is undefined under saturation.
Shortest-processing-time optimality, requiring an identical makespan	A reordering policy that changes how much work is done rather than only when it is done.
EASY backfill [11], carrying its documented negative case under honest over-estimates	An implementation that silently enforces runtime estimates, which real clusters do not.
Invariants: gang atomicity, chip-hour conservation as a per-flush accounting identity, per-flush allocation accounting, quota caps with labelled demotions, reservation exclusivity, the exact-microsecond crossing-penalty identity, byte-identical output for one scenario and seed on one platform	The bug class that yields plausible numbers from an internally inconsistent fleet state.

schedulers needs 24 more replays. An open question, not a claim.

5.4 Closed-form checks

Trace replay says whether the simulator matches one real cluster; closed-form checks say whether the engine underneath computes what queueing theory already knows. The checks form a ladder, from pure algebra at the bottom to a real trace at the top, and the repository calls each step a *rung*; we keep its word.

Two rungs need their notation unpacked. Queueing theory labels a system by its arrival process, its service-time distribution and its server count: M/M/c means memoryless arrivals, memoryless service times and c servers, and M/G/1 means memoryless arrivals, a general service-time distribution and one server. For a handful of such systems the mean wait has an exact formula — Erlang-C for M/M/c, Pollaczek–Khinchine for M/G/1 — so we can build the same system in fleetsim, run it, and check the simulated mean wait against the algebra.

Each rung in Table 6 runs in continuous integration and is chosen for a specific failure it would catch. Not everything planned is shipped, and the unshipped items are close cousins of shipped ones, so they are worth naming precisely. The M/G/1 conservation-law identity — a statement that a certain weighted sum of waits is the same under any non-anticipating discipline, and a different object from the chip-hour accounting invariant in the table — is a documented target. So is the M/M/c generalisation of the two-tier priority rung, whose shipped form is single-server. So are most distributional self-checks on the traffic generator.

Table 7: Anti-goals: published results fleetsim deliberately does not reproduce.

Published number	Why not	Disposition
Philly $\sim 52.3\%$ GPU utilization [7]	A hardware cycle counter, not scheduler occupancy. A discrete-event simulator produces <i>allocation</i> occupancy: a different and higher quantity.	Out of scope
Philly fair-share fragmentation-delay split [7]	Needs per-job delay-cause attribution, which fleetsim does not compute.	Still deferred
Helios QSSF column [5]	Needs a job-name column absent from the public CSV plus a learned duration predictor; the SJF-oracle is the reproducible upper-bound proxy.	Out of scope
Alibaba 50% GPU-sharing saving, median 0.042 GPU per instance [22]	Needs fractional, sub-chip GPU allocation; fleetsim shares only whole chips within a node.	Still deferred
Borg absolute occupancy and JCT [20]	Resources are normalized to $[0, 1]$, the data is query-only, and it covers eight independent 12,000-machine cells.	Never a replay target
CPU contention, co-location interference	Not modelled: an analytical speed model with no memory-bandwidth contention.	Out of scope

5.5 What fleetsim deliberately does not reproduce

Table 7 lists the published numbers fleetsim does not try to match. They are stated so that nobody quotes this paper for something it did not test; none is asserted anywhere in the suite, so none can surface as a test failure, which is precisely why they must be written down. Two rows deserve a note: the fragmentation-delay split and fractional-GPU sharing were both targets for the release that then spent its budget on placement, and are recorded as still open rather than quietly re-dated. Separately, the Philly status-split rung, though written to specification, has never been run against the real one-gigabyte trace in this build — the artifact is stored with Git LFS and a plain fetch returns a pointer file. Published targets, tolerances and structural assertions on a small synthetic slice exist, but there is no measured Philly number here, and the suite marks those rows as unmeasured rather than as agreement.

6 Case studies

Three studies follow, each shipping as a runnable example directory and each a paired experiment: runs differing by exactly one configuration line. Because every random draw comes from a stream keyed by name rather than call order, every other class’s arrivals, sizes and durations stay byte-identical across the pair.

6.1 Frontier scale: running hot without starving production

`examples/04_frontier/` is $4 \text{ clusters} \times 32 \text{ pods} \times 16 \text{ scalable units} \times 32 \text{ nodes} \times 8 \text{ chips} = 524,288$ H100 GPUs, run for two simulated days with a 120s scheduler round at seed 42. Node failures are off deliberately: a 16,384-node gang is eight times the size of Llama 3’s 405B run, which measured one unplanned interruption every 3.1 hours [12], so it would break about every 23 minutes (§2.4 gives the derivation and the disagreement between the two published anchors), and the restart churn

	No backlog	With backlog
Occupancy (window)	0.477	0.995
Goodput (window)	0.966	0.880
Productive fraction, occupancy $\times$ goodput	0.461	0.876
Eval queue wait, p50	61.3 s	77.8 s
Batch fine-tune queue wait, p50	66.9 s	116.4 s
Pretrain queue wait, p50	51.7 s	163.8 s
Eval queue wait, p99	236 s	1,292 s
Preemptions per minute	0.068	0.565
Jobs finished	23,538	34,422

Table 8: A 524,288-chip fleet with and without a preemptible backlog, seed 42. The first three rows are measured over the steady-state window (the middle 80 % of the run); the rest are full-run figures. The two scopes are not interchangeable, which is why the example labels them.

would drown the scheduling effect. This is therefore a failure-free result. The two runs differ only in whether a best-effort class exists: the lowest, always-preemptible priority band, in the sense Borg made standard [20]. A closed loop holds a fixed number of those jobs in flight; over the two-day horizon that cycles 11,252 of them through the fleet.

Table 8 is the trade. The backlog roughly doubles steady-state occupancy and finishes 10,884 more jobs. Production medians move by one or two scheduler rounds — the largest is the pretrain median, 51.7 s to 163.8 s — which is the sense in which production waits stay roughly flat; the tails do not, the eval p99 going from 236 s to 1,292 s, roughly 4 minutes to 22. The cost is goodput, 8.6 points of it: the backlog runs with checkpointing off, which is exactly what makes it instantly reclaimable, so every eviction discards a whole *stint* — one contiguous period of holding an allocation, and with no checkpoint behind it, everything that job computed since it was last given chips. The product of the two still nearly doubles. Best-effort jobs are excluded from every wait distribution by contract, waiting time being meaningless for a closed loop at saturation.

Then one large job: a 131,072-chip pretraining gang of 32 pod-sized segments, modelled on the multi-pod shape Llama-3 405B training used [12], submitted at hour 20.57 of both runs. On the idle fleet it waits 113 s; above 99 % occupancy, 233 s — exactly one 120 s round more, so the reclaim costs nothing and the wake boundary costs everything. One wake carries 345 preemptions, allocated chips dip from 523,946 to 406,406 and recover to 522,998 a round later, and pending jobs spike from 116 to 441. The gang spans 32 pods in both runs.

6.2 Occupancy and goodput can move in opposite directions

`examples/05_topology_tradeoff/` is 8,192 H100s (8 pods $\times$ 16 racks $\times$ 8 nodes $\times$ 8 chips) over seven days at seed 42, failures off, near 86 % utilization so pod-shaped holes are scarce. Pretraining jobs draw 256, 512 or 1,024 chips and ask for one pod, but softly: they wait ten minutes for a pod-shaped hole, then take whatever fits. Run A charges a job spanning more than one pod a speed multiplier of 0.7, so it needs $1/0.7 \approx 1.43$ times the wall time for the same work; run B sets that multiplier to 1.0.

Switching the penalty *on* raises occupancy, 0.826 to 0.862, and *lowers* goodput, 0.983 to 0.861. Both are correct: occupancy is the fraction of healthy chip-time allocated to some job, goodput the

	A: crossing costs $0.7\times$	B: crossings free
Occupancy (window)	0.862	0.826
Goodput (window)	0.861	0.983
Cross-pod placements	35 of 112	17 of 112
Pretrain queue wait, p50	484.5 s	47.9 s
Pretrain job completion time, p50	51,002 s	43,136 s
Pretrains completed	99	103
Eval queue wait, p50	31.9 s	31.4 s

Table 9: Turning the cross-pod speed penalty on raises occupancy and lowers goodput at the same time (seed 42, 7-day horizon).

fraction of that allocated time which produced surviving work. A slowed job holds its chips about 1.43 times longer for the same work, inflating the first and deflating the second. The busier-looking run completed four fewer pretrains and finished them later.

Table 9 also shows the feedback loop that makes this self-reinforcing, and it reads backwards at first: charging for pod crossings *doubles* them, from 17 of 112 placements to 35. The chain is that a penalised job holds its chips 1.43 times longer, so pod-shaped holes are scarcer when the next pretraining job arrives, so more jobs exhaust their ten-minute soft wait and take whatever fits. Pretrain queue wait moves accordingly, 47.9 s to 484.5 s.

Two details keep this honest. Run B’s goodput is 0.983 rather than 1.0 because of checkpoint-save amortization — a one-hour interval with a 60 s save costs about 1.6 % — so the penalty accounts for 12 of those points, not 14. And the effect is confined to large jobs: eval waits are 31.9 s and 31.4 s, while the chance of crossing pods rises from 18 % at 256 chips to 61 % at 1,024. This is the strongest argument available against an occupancy-only dashboard: occupancy rose 3.6 points and the fleet got worse.

6.3 Placement: where the small jobs land decides how long the big ones wait

`examples/07_placement_study/` is deliberately tiny — 4 racks $\times$ 8 nodes $\times$ 8 chips = 256 H100s in 32 nodes — and deliberately two levels deep, since `best_fit` and `consolidate` are bit-identical on a single-level fleet. Fourteen days, seed 42, failures off. The workload is mice (1-, 2- and 4-chip jobs lasting hours) plus gangs of 16, 32 and 64 chips — 2, 4 and 8 *whole nodes* — at an offered load of 78 % by construction, 81 % realised at this seed, because Poisson arrivals and a lognormal duration tail land above the mean here. A best-effort FIFO scheduler holds ordering fixed, so only the placement policy varies. It reproduces the mechanism the Helios replay identified: a 1-chip job landing on a completely free node converts eight whole-node chips into a remainder no gang can claim, and the gangs, not the mice, pay for it.

Tightest-fit packing (`best_fit`; `consolidate` is within noise of it here) cuts time-average stranded nodes from 6.55 to 5.31 of 32 and the mean queue wait of an 8-node gang from 19,056.8 s to 16,195.3 s (Table 10). The mice pay for it: their median wait is flat, 31.0 against 31.4 s, but their mean rises from 80.7 s to 95.3 s — about 15 seconds of mouse wait buying about 2,900 seconds of gang wait. `spread` prices the mechanism from the other side: it strands 25.31 of 32 nodes, never starts 80 of the 109 8-node gangs and delivers 28.4 % fewer chip-hours on identical hardware — and it averages only the gangs it did start.

	<code>first_fit</code>	<code>best_fit</code>	<code>consolidate</code>	<code>spread</code>
Stranded whole nodes, mean (of 32)	6.55	5.31	5.37	25.31
64-chip gang queue wait, mean (s)	19,056.8	16,195.3	16,314.7	739,842.1
64-chip gangs ever started (of 109)	107	107	107	29
Mice queue wait, p50 (s)	31.0	31.4	31.3	30.1
Occupancy (full run)	0.8141	0.8141	0.8141	0.5829
Jobs completed	3,436	3,436	3,436	3,309

Table 10: Four placement policies on identical hardware, identical arrivals and identical ordering (seed 42, 14-day horizon). A “stranded” node is a healthy node that is partly occupied, so its free chips cannot satisfy any whole-node request. The whole study runs in about 10s on a laptop: about 1s each for the three packing arms and about 7s for `spread`, whose queue is roughly 30× deeper.

The negative result matters more. Full-run occupancy, goodput and allocated chip-hours are *bit-identical* across `first_fit`, `best_fit` and `consolidate` — 0.8140502977503281, 0.9831464030337717 and 70,021.35041129222 chip-hours — because the same 3,436 jobs complete for the same durations in all three, and windowed occupancy moves 0.2 percentage points. The entire placement effect is *when* work ran, never how much: a placement study reporting only occupancy has measured almost nothing.

Three caveats travel with it. At seed 1234 the mechanism metric still improves by about 20% while the outcome regresses: mean gang wait 8,205s to 9,407s, 15% worse. A hard `within: rack` constraint inverts the ranking outright — 64-chip mean wait 39,677s under `first_fit` against 121,486s under `best_fit` — because node-level tightest-fit spreads gangs across racks, so no rack ever drains. And this scheduler is best-effort, so a stuck gang does not hold up the mice behind it: a floor, not a best case. “`best_fit` is a better scheduler” is not a claim this study supports.

7 Implementation, testing and reproducibility

Language and size. `fleetsim` is Python 3.11 or newer under Apache-2.0, depending at runtime on `numpy`, `pandas`, `pyarrow`, `pyyaml` and optionally `matplotlib`. The event loop is hand-written rather than built on a simulation framework — full control over tie-breaking and determinism is worth about 200 lines — and the local web application is pure standard-library `http.server`, so running it adds no dependency. The simulator is 22,563 lines of Python across 47 files; the tests are 18,896 lines across 36 files plus 2,585 lines of validation rungs across 12, and `pytest` collects 1,024 of them. The four placement policies behind Section 6.3 are 340 lines together; the FIFO scheduler is 48.

Determinism, and its per-platform limit. Simulated time is a 64-bit integer count of microseconds, so ordering and tie-breaking are exact. Random draws come from `numpy` seed sequences whose spawn key is the first 16 bytes of the SHA-256 of the stream’s *name*, never Python’s salted `hash()`, so a given stream yields the same numbers in every process on every machine. The contract is that an identical scenario and seed produce byte-identical Parquet and JSON, on a given platform, and a CI test enforces it by running the whole pipeline twice with failures, maintenance and preemption switched on.

That qualifier is load-bearing. The lognormal and exponential samplers call the platform’s C math library, whose `exp` and `log` differ in the last bit between `glibc` and Apple’s implementation.

An early release (v0.1; the project is at v0.8) shipped a compatibility test pinning a SHA-256 of the arrival stream captured on macOS, and it failed the first time CI ran on Ubuntu. The replacement compares golden *data* — the first 1,000 arrivals at nine significant digits, plus aggregate counts — with tolerances: 2 microseconds on times, 10^{-6} relative on durations, and at most 5 differing rows per 1,000 for draws sitting on a discrete knife edge. Cross-platform reproduction means the same distributions, not the same bytes.

Continuous integration, and what a clean machine caught. One GitHub Actions workflow runs on every push and pull request: Ubuntu, a Python 3.11 and 3.12 matrix, an editable install of the package and the out-of-tree plugin example, `pytest tests validation -q`, then a smoke test running `fleetsim validate` and `fleetsim run` on the minimal example. It runs with no network in about 80s with exactly four skips — two Helios full-trace rungs, one Philly, one frontier-budget gate — and anything else skipping is a signal.

Two defects surfaced only because CI is a clean room. The first was an environment leak: a test asserting that every shipped example validates passed locally only because the developer’s virtual environment still had the example-03 plugin scheduler installed from an earlier session, and that example names an out-of-tree scheduler which enters the registry only when its package is installed. The fix skips an example whose scheduler is unregistered — validation *should* reject unknown names — while asserting a floor on how many examples were actually checked, so the skip cannot hollow the test out; CI now installs the plugin. The second was a concurrency deadlock a laptop could not reach: the 3.12 job hung at 91 % and was killed by the job timeout while 3.11 passed the same commit, the developer machine having many cores and the hosted runners two. The cause was a helper process that starts lazily, combined with a setup step that re-ran on every worker-pool creation and mutated interpreter-global state; on a two-core runner the two raced. Building that context once per process fixed the hang, and a fault handler timeout was added so that the next hang of any kind would fail loudly with every thread’s stack. That dump then named a deeper bug: CPython invokes a future’s done-callback while holding the executor’s shutdown lock, so a callback calling `shutdown()` deadlocks itself. A broken pool is now disposed on a separate daemon thread.

Adversarial review. Several releases were reviewed adversarially — a pass whose goal was to find defects rather than approve — and each such pass leaves a regression file that lists its findings: `test_review_fixes.py` in `tests/`, plus its v02, v04, v05 and v05_hardening successors. Five files across eight releases, so this is a habit rather than a gate. The same discipline governs the published figures: each lives once, as data, in a single module from which the validation tests import their assertion constants and which the web application serves verbatim, and a test checks by object identity that nobody has retyped a number into a test.

Reproducing the validation. The offline checks are one command, `pytest tests validation -q`, and `fleetsim validation run` prints the vendored-slice checks in about 5s. The trace results are opt-in and environment-gated because they download real data: `FLEETSIM_HELIOS_FULL=1` `pytest -m trace_full validation/test_helios_ratio.py` fetches the 36 MB Helios archive [5] and replays four clusters under two schedulers in 266s. The Philly rung [7] is gated the same way but needs `git lfs pull` first, since a plain fetch of that 1 GB artifact yields a 135-byte pointer; it was written to specification and has not been run against the real trace in this build. Each rung self-skips unless its variable is set and never downloads implicitly. `fleetsim validation cite` prints each trace’s required attribution.

Tooling. Six subcommands cover the loop: `fleetsim validate`, `run`, `compare`, `plot`, `viz` and `serve`. Two properties of them carry an argument rather than a feature list. `viz` renders a run directory into one self-contained HTML replay that makes zero external requests, and a test greps both the template and the output for any external URL or fetch call, so the claim is enforced rather than intended. And parameter sweeps run one simulation per grid cell in worker processes: the same sweep run twice is byte-identical cell for cell, so parallelism does not weaken determinism, and a test enforces that too.

8 Related work

Recall from §1 that simulators are best compared by *altitude*: what one simulated run represents. A run of `fleetsim` represents weeks of a whole fleet — many clusters, many tenants, thousands of jobs arriving, queueing, running, being preempted and occasionally dying with their nodes. Most existing tools sit at a different altitude, which is not a deficiency in them: a tool that models one job’s network traffic in microseconds is not a worse fleet simulator, it is not a fleet simulator at all, and it answers questions `fleetsim` cannot. Table 11 summarises the landscape. One caveat: this is a survey of what each tool’s papers and documentation describe, not a benchmark — we did not install, run or measure any of these systems.

Single-cluster scheduler simulators. Blox [4] and the simulators built inside scheduling papers such as Gavel, Pollux and Sia take a job trace and a policy, apply them to one cluster of N interchangeable GPUs, and report completion times. They are the closest prior art, and `fleetsim` borrows from them directly: Blox’s decomposition of a scheduler into an *ordering* policy (who runs next) and a *placement* policy (where it goes) is the API we adopted, and its convention of measuring the steady middle of a run is our metrics window. What a flat GPU count cannot express is a job failing to fit for structural reasons. Our placement study makes the consequence concrete: on a 256-chip fleet at one seed, changing only the placement rule left occupancy identical to the last digit (0.8141 under three of four policies) while the mean queue wait of eight-node gangs moved from 19,057s to 16,195s. A simulator whose capacity is an integer cannot represent that mechanism; node failures are the second gap.

Per-job network simulators. ASTRA-sim [2] and SimAI [17] simulate one training job’s iteration in detail — collectives, fabric topology, congestion. This is precisely the layer `fleetsim` collapses into a single number: a placement spanning a configured topology level multiplies the job’s progress rate by a configured multiplier, and there is no default multiplier because the honest value is fleet-specific. The two altitudes compose rather than compete — a network-level study is how a lab would derive that multiplier. What these tools do not have is a queue: one already-placed job has nothing to wait behind.

Inference-serving simulators. Vidur [21] and LLMervingSim model a serving deployment: request arrivals, batching, KV cache, latency percentiles. `fleetsim` models inference at replica granularity — a replica being a fixed block of chips held for a service — and stops there, because per-request burstiness sits below fleet altitude.

HPC batch simulators. Batsim [3] and the Slurm simulator carry decades of batch-queue semantics, and `fleetsim` copies Batsim’s central architectural choice: the engine mutates state, policies emit intents, and the engine validates and applies them, in-process rather than over a socket. EASY

Table 11: Related tools by altitude. Characterisations are taken from each tool’s published description, not from measurements we made.

Family	Altitude	What it does well	What a fleet question needs that it omits
Blox [4]; the simulators inside Gavel, Pollux, Sia	One cluster	Fast, faithful replay of published deep-learning scheduling policies; Blox reports seven schedulers in 12–1157 lines each	Capacity is a flat GPU count: no topology levels, no node failures or repair, no inference, no multi-cluster
ASTRA-sim [2], SimAI [17]	One job $\times$ network	Collective communication and per-iteration time at packet and flow detail	There is no queue: the job is already placed, so there are no arrivals, no waiting and no preemption
Vidur [21], LLMservingSim	One inference service	Request batching, KV-cache behaviour and tail latency for LLM serving	Inference only, one service; training gangs, quota and failures are outside the model
Batsim [3], the UBCCR Slurm simulator	HPC batch	Decades of batch-queue semantics, backfill [11], and a clean engine/policy separation	No accelerator cost model: preemption is not priced in lost training work, and there is no checkpoint or gang-restart notion
AIRSim [1]	Reliability of one job	Failure, repair and restart accounting for a single long training run	No scheduler, no queue, no competing tenants
kube-scheduler-simulator [10]	Kubernetes placement	Explains, decision by decision, why a pod landed where it did	Wall-clock, no simulated time: no job completion time, no occupancy over a horizon
Borg [20, 19], MAST [14], Singularity [18]	Fleet — the existence proof	Run real fleets at exactly this altitude and publish what the hard problems are	They are production systems, not simulators; there is no public artefact in which to run an experiment

backfill [11] is a shipped `fleetsim` scheduler for the same reason. The gap is not gangs as such — MPI jobs are gangs — but the accelerator cost model around them: killing a training job destroys the work done since its last checkpoint, a GPU job cannot be suspended in place because its memory is held, and a restart costs minutes. Those costs are what make preemption policy interesting, and what these models do not price.

Production fleet managers. Borg [20, 19], MAST [14] and Singularity [18] operate at fleet altitude, and supply much of `fleetsim`’s substance: Borg’s priority bands are the tier semantics, and published reliability figures [15, 12] set the failure defaults. Borg TNG’s tail of per-job *resource-hours* ($\alpha \approx 0.69\text{--}0.77$, the top 1% of jobs accounting for 97–99% of load) is a target the traffic generator is meant to satisfy — not a configured parameter, and not yet checked by a shipped rung.

What none of them ships is something you can run, and the public traces [7, 5, 22, 6] record only what happened under one policy, never what a different one would have done. That counterfactual is the gap. One precision about the borrowing: `fleetsim`’s quota model is inspired by MAST but is not a model *of* it (Section 9).

The workload generator likewise stands in the workload-modelling tradition of rigid parallel jobs [13], with non-Poisson arrivals [16] and lognormal duration bodies taken from the Helios and Acme characterizations [5, 6] — and the closed-form checks of Section 5 are ordinary queueing theory [9]. A cross-simulator check against Blox’s published percentiles was planned and never built.

9 Limitations and future work

No fractional GPUs. The smallest unit `fleetsim` allocates is one whole chip. Alibaba PAI’s headline results [22] — roughly a 50 % saving from GPU sharing, with a median instance requesting 0.042 of a GPU — are therefore out of reach, and no placement policy substitutes for the missing mechanism. It was a target of the previous release that the release did not reach, and we record it as still open rather than quietly re-dating it.

No per-job delay-cause attribution. `fleetsim` records how long each job waited, not *why*. Philly’s split of queueing delay into a fair-share component and a fragmentation component [7] — the result that motivated this design, where fragmentation accounts for 59–98 % of multi-GPU delay — cannot be reproduced, and is a second promise carried over from the release before last, still open. The metric we did add, a count of partially-occupied nodes, is a down payment, not a substitute, and it is directional rather than proportional: an 8.9 % reduction in its time-average accompanied a 25.7 % reduction in mean FIFO completion time, because a mean over all instants dilutes exactly the moments when the queue head needs a whole free node.

The performance model is analytical. A job’s progress rate is the product of configured per-level multipliers. The identity $completion = start + overhead + remaining\ work / rate$ is pinned exactly in integer microseconds, but the rate is an input, not a derivation, and there is no default crossing penalty. Chip heterogeneity is worse off: the throughput hook always returns 1.0, and scenarios using unpinned chip types are rejected outright. CPU contention and co-location interference are not modelled at all.

Failures are sampled from a rate, not from a physical model. One exponential draw at the fleet-wide aggregate rate, a thinning test, then a uniform victim pick — at a default mean time between failures of 42 node-days with a fixed cause mix (60 % GPU/HBM, 10 % network, 13 % software, 17 % other). Nothing in that is physical: no correlated failures (a rack losing power takes one node at a time here), no thermal or age dependence, and unreliable “lemon” nodes are off by default. Spot preemption is deliberately harsher than the products it abstracts — zero notice, no banked progress — so spot losses are an upper bound, not an estimate.

One trace family, validated in depth. The quantitative validation is Helios [5]: four clusters, both schedulers, the full September window. Philly [7] is exercised only at converter level, and even there the full check, though written, was never run against the real one-gigabyte artefact, so every Philly row reports “unmeasured” rather than a number; only structural assertions on a synthetic 2,000-row slice run in CI. Alibaba PAI is registered but never replayed. One trace family is one lab’s workload, one scheduler lineage, one hardware generation.

The absolute numbers are order-sensitive. On Saturn, 35.5% of the 105,969 jobs in the window share an exact submit second, and 92.9% share a 60-second scheduler round — so the order in which tied arrivals are served is a modelling choice, not a property of the trace. Flipping the FIFO tie-break from ascending to descending job id moves Saturn’s mean FIFO completion time from 75,329s to 62,549s, a 17.0% move, and inverts which placement policy looks closer to published. Ascending id is demonstrably the faithful order (ids are assigned at submission and are monotone in submit time on three of the four clusters), so this is a sensitivity probe under a deliberately wrong order, not an error bar. Its magnitude still matters: 17% is wider than the $\pm 14\%$ agreement we report on absolute completion times, and a large fraction of the 25.7% move the placement change itself produced. No single cluster’s absolute number can select a modelling choice, which is why our discriminator aggregates four clusters — and whether that discriminator survives the same perturbation is *unmeasured*: the probe ran on one cluster under one scheduler (Saturn under FIFO, in both tie-break orders), and extending it to all four clusters under both schedulers needs 24 more replays.

Single-metro scheduling only. There is one scheduling stage over one metro. Two-stage scheduling across metros — the MAST shape [14], where a first stage decides which region a job belongs in — is schema-present and rejected as not implemented, as are TPU optical-switch predicates, multi-gang jobs and autoscaling inference. The quota model is also simpler than the systems that inspired it: *fleetsim* commits quota at admission against queued demand and demotes irreversibly, whereas MAST and comparable systems evaluate against running usage at scheduling time and treat over-quota work as borrowed capacity that can be handed back. The quota scenario shipped as *examples/06_economics/* is not among the case studies of §6; its demotion counts are partly an artefact of that admission-time choice, which is why we do not quote them here.

Smaller ones, stated plainly. The backfill scheduler implements the conservative one-rule form of EASY, so measured backfill rates undershoot published EASY reproductions. Byte-identical output holds only within a platform, since the floating-point samplers differ in the last bit across operating systems. And the performance envelope in the design document — six simulated months of a 100,000-GPU fleet — is a target, not a measurement: what has been measured is six months of a 2,048-chip fleet in about nine minutes, and two days of a 524,288-chip fleet in about eighty seconds, on a laptop.

Future work. Five items, in priority order. *Fractional GPUs*, the single change that unlocks a whole trace family’s published results. *Per-job delay-cause attribution*, which would reproduce Philly’s split and turn the stranding metric into a per-job account of where a wait came from. *More trace families* — the Philly artefact, then Alibaba PAI — because one validated family is the weakest part of Section 5. *TPU torus geometry*: TPU pods wire their chips into a three-dimensional torus — a grid whose edges wrap around — and a job is given a rectangular sub-block of it, a *slice*. Feasibility then depends on which free *cubes* (contiguous $4 \times 4 \times 4$ blocks) exist rather than on how many chips are free, and the published optical-switch result removes contiguity constraints for cube-multiple slices [8]. And a *throughput matrix for heterogeneous chips*, replacing the hook that returns 1.0, so a fleet mixing chip generations can be modelled rather than approximated as uniform.

10 Conclusion

We built an open-source discrete-event simulator that operates at fleet altitude: a hierarchy of clusters, pods, racks and nodes; gang scheduling with atomic, immutable allocations; priority preemption priced in lost work; node failures, repair and maintenance drains; and a generative traffic model drawn from published trace characterisations.

We then tried to break it. Against the four Helios clusters [5], `fleetsim` reproduces the published ratio between FIFO and shortest-job-first mean completion times with a mean absolute error of 3.4%, and lands within $\pm 14\%$ of the published absolute completion times — a weaker, reported-not-asserted result the repository deliberately calls “right ballpark” rather than reproduction. Both hold only under three modelling choices we name every time we state the result, and inside tolerance bands wide enough that a deliberately bad placement policy passes them too. The engine also reproduces four textbook queueing results it could have failed — Erlang-C, Pollaczek–Khinchine, two-class preemptive-resume priority and shortest-processing-time optimality — plus a behavioural property test of EASY backfill. And we have written down what it does not reproduce, and why.

If one sentence survives: matching a published table was not the useful part — the useful part was that matching it forced three modelling choices into the open, and that occupancy, the metric everyone reports, was bit-for-bit identical across placement policies whose queueing behaviour was not.

References

- [1] AIReSim: a reliability simulator for a single large training job. No venue, year or identifier: the repository names this tool as the source of its node-repair defaults (automatic repair uniform on 60–180 minutes, manual repair 1–3 days for 10% of failures) but does not pin it to a citable artefact, so those two defaults are the least checkable in the paper. Stated here rather than dressed up.
- [2] ASTRA-sim: a distributed machine-learning system simulator. <https://github.com/astra-sim/astra-sim>, 2023.
- [3] Batsim: a realistic language-independent resources and jobs management systems simulator. Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP), 2016.
- [4] Blox: A modular toolkit for deep learning schedulers. European Conference on Computer Systems (EuroSys ’24), 2024. <https://github.com/msr-fiddle/blox>.
- [5] Hu et al. Characterization and prediction of deep learning workloads in large-scale GPU datacenters. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’21)*, 2021. DOI 10.1145/3458817.3476223, arXiv:2109.01313. Trace: <https://github.com/S-Lab-System-Group/HeliosData>, CC-BY-4.0.
- [6] Hu et al. Characterization of large language model development in the datacenter. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI ’24)*, 2024. The “Acme” characterization; arXiv:2403.07648.
- [7] Jeon et al. Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *USENIX Annual Technical Conference (ATC ’19)*, 2019. Trace: <https://github.com/msr-fiddle/philly-traces>, CC-BY-4.0.

- [8] Jouppi et al. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *International Symposium on Computer Architecture (ISCA '23)*, 2023. arXiv:2304.01433.
- [9] Kingman. The single server queue in heavy traffic. *Mathematical Proceedings of the Cambridge Philosophical Society*, 1961.
- [10] kube-scheduler-simulator. <https://github.com/kubernetes-sigs/kube-scheduler-simulator>. Kubernetes SIG Scheduling.
- [11] Lifka. The ANL/IBM SP scheduling system. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, 1995. The EASY backfill algorithm. fleetsim’s documented guarantee is the later form of Mu’alem and Feitelson, IEEE TPDS, 2001: the queue head never starts later than the shadow computed when backfill was granted.
- [12] The Llama 3 herd of models. arXiv:2407.21783, 2024.
- [13] Lublin and Feitelson. The workload on parallel supercomputers: Modeling the characteristics of rigid jobs. *Journal of Parallel and Distributed Computing*, 2003.
- [14] MAST: Global scheduling of ML training across geo-distributed datacenters at hyperscale. *USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, 2024.
- [15] Revisiting reliability in large-scale machine learning research clusters. arXiv:2410.21680, 2024. Source of fleetsim’s 42-node-day node MTBF, 15-minute restart overhead and one-hour checkpoint-interval defaults.
- [16] Paxson and Floyd. Wide-area traffic: The failure of Poisson modeling. *IEEE/ACM Transactions on Networking*, 1995.
- [17] SimAI: a large-scale AI training simulation toolkit. <https://github.com/aliyun/SimAI>, 2025.
- [18] Singularity: Planet-scale, preemptive and elastic scheduling of AI workloads. arXiv:2202.07848, 2022.
- [19] Tirmazi et al. Borg: The next generation. In *European Conference on Computer Systems (EuroSys '20)*, 2020.
- [20] Verma et al. Large-scale cluster management at Google with Borg. In *European Conference on Computer Systems (EuroSys '15)*, 2015.
- [21] Vidur: A large-scale simulation framework for LLM inference. *Proceedings of Machine Learning and Systems (MLSys '24)*, 2024.
- [22] Weng et al. MLaaS in the wild. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI '22)*, 2022. Alibaba PAI trace: <https://github.com/alibaba/clusterdata>.